

Build Task — Project 4: Multi-Agent System — Hints & Solution

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper LangGraph, including how a real production multi-agent pipeline handles the messy edge cases). Read Basic first even if you already know LangGraph — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — What you're building, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)
- [Solution](#)

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — What you're building, and the exact pieces

Basic Version

You're building 5 small pieces that hand work to each other, in this order: a **supervisor** that decides who goes next, a **research agent** that looks things up, an **analysis agent** that figures out what the research means, a **writer agent** that drafts the final piece, and a **reviewer agent** that checks the draft and can send it back for changes.

The supervisor's job is simple: look at what's been done so far (state), and decide who runs next. The reviewer's job is the one with a twist: it can say "not good enough, try again," but only up to a limit — never forever.

Here are the exact pieces you need to look up and use:

- `from langgraph.graph import StateGraph, START, END`
- `from langgraph.types import Command` — every agent node returns one of these: a state update, plus who runs next.
- A `TypedDict` (or similar) holding the shared state every agent reads and writes.
- A `revision_count` field in state, and a `MAX_REVISIONS` constant, checked every time the reviewer rejects.
- A `routing_log: list` field, so the final output can show exactly which agents ran, in what order.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Think about the shape first, before any code: `supervisor → research_agent → analysis_agent → writer_agent → reviewer_agent`, where every specialist node hands control back to the supervisor when it's done, and the supervisor decides the next step from what's now in state. This is the same "one router, specialists report back" shape `supervisor_compare` already built — Project 4 just has 4 specialists instead of 2, and one of them (the reviewer) can send work backward, not just forward.

The key design decisions, matching the Build Task's own Requirements:

- **Shared vs. private state** — per Core Concepts' Shared state design section, state should hold each agent's *finished* output (`research_findings` , `analysis` , `draft` , `review_feedback`) and routing/status info (`revision_count` , `routing_log`), not any agent's raw internal reasoning.
- **The reviewer's decision** — it reads `draft` (and maybe `analysis` , to judge if the draft actually reflects it), and returns either `Command(goto=END)` with an "accepted" status, or `Command(goto="writer_agent")` with feedback, incrementing `revision_count` either way.
- **The hard limit** — checked in the reviewer node itself, before it even asks "is this good," the same defensive-check-first pattern `convergence_and_cost_cutting` practiced: if `revision_count >= MAX_REVISIONS` , stop and report "couldn't agree," full stop, regardless of what the reviewer would have said.
- **No redundant tool calls** — a Constraint from the README: each specialist's node should run exactly once per sub-task unless the reviewer specifically routes back to `writer_agent` ; nothing should call `research_agent` twice for the same task.

Sketch `state.py` 's shape and all 5 node functions' signatures, then go one step further: think past "route to the next agent and check a counter" — ask **what does "a visible log of why the supervisor routed the way it did" (from the README's own Outputs section) actually require, beyond just a list of node names?** A bare `["research", "analysis", "writer", "reviewer"]` log tells you *what* ran, but nothing about *why* the supervisor thought that was the right next step — which is exactly the gap `ambiguous_routing` found matters once a routing decision isn't perfectly obvious.

The same question applies to the revision loop: the README's Test Cases table specifically requires a **clear** "couldn't agree" report when revision never settles, not just a stopped graph. That means carrying forward every rejection reason, not just the count — the exact pattern `convergence_and_cost_cutting` built, applied here for real.

Two extra design pieces answer both questions:

- **A routing log entry that includes the *reason*, not just the node name** — e.g. `{"step": "research_agent", "reason": "no research_findings in state yet"}` instead of just `"research_agent"` — so the final output can actually explain the supervisor's decisions, satisfying the Build Task's Outputs requirement directly.
- **Full LangGraph saved-state (a checkpoint), carried over from Project 3** — per the Build Task's own Requirements, wire in `from langgraph.checkpoint.memory import MemorySaver` (or a persistent equivalent) and compile with `checkpointer=memory`, passing a `thread_id` in config — this is what lets a run be paused, inspected, or resumed, the same saved-state discipline Doc10/Project 3 already established, now carried through a 4-agent graph instead of a single agent.

```
from langgraph.checkpoint.memory import MemorySaver

memory = MemorySaver()
graph = builder.compile(checkpointer=memory)

config = {"configurable": {"thread_id": "task-1"}}
result = graph.invoke(initial_state, config=config)
```

Difference between Basic and Intermediate: Basic names the 5 agents, the hand-off shape, and the exact LangGraph pieces (`Command`, a shared state `TypedDict`, a revision counter) for a first working pipeline. Intermediate maps each Build Task Requirement onto a specific design decision — what's shared vs. private in state, where the hard limit gets checked, why redundant tool calls need to be actively prevented — then closes the gap between "it works" and what the Build Task's own Outputs and Requirements actually demand — a routing log that explains *why*, not just *what*, and real saved-state via a checkpoint, carried over from Project 3 as the Requirements explicitly ask for.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
state.py:
    SharedState holds: task, research_findings, analysis, draft,
    review_feedback, accepted, revision_count, routing_log

agents/supervisor.py:
    look at what's already in state, decide who's next:
        no research_findings -> research_agent
        no analysis -> analysis_agent
        no draft -> writer_agent
        draft but not accepted, and revisions left -> reviewer_agent
        else -> done (either accepted, or out of revisions)

agents/research_agent.py, analysis_agent.py, writer_agent.py:
    each does its one job, fills in its field, goes back to supervisor

agents/reviewer_agent.py:
    check the draft
    if good enough: accepted = True, done
    if not, and revisions left: bump revision_count, send back to writer_agent
    if not, and no revisions left: accepted = False, done, with a reason

graph.py:
    wire all 5 nodes into a StateGraph, entry point = supervisor

main.py:
    run graph.invoke on a task, print routing_log and the final result
```

Expected output if you run just this (nothing calls the graph yet): nothing — a graph isn't invoked until `main.py` calls `graph.invoke(initial_state)` with a real task. Build the initial state dict and call it to see the pipeline actually run end to end.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

state.py

```
from typing import TypedDict

class SharedState(TypedDict):
    task: str
    research_findings: str
    analysis: str
    draft: str
    review_feedback: str
    accepted: bool
    revision_count: int
    routing_log: list
```

agents/supervisor.py

```
from langgraph.types import Command

MAX_REVISIONS = 3

def supervisor(state):
    log = state["routing_log"]
    if not state.get("research_findings"):
        return Command(
            update={"routing_log": log + ["research_agent"]},
            goto="research_agent",
        )
    if not state.get("analysis"):
        return Command(
            update={"routing_log": log + ["analysis_agent"]},
            goto="analysis_agent",
        )
    if not state.get("draft"):
        return Command(
            update={"routing_log": log + ["writer_agent"]},
            goto="writer_agent",
        )
```

```

if state.get("accepted"):
    return Command(goto="__end__")
if state["revision_count"] >= MAX_REVISIONS:
    return Command(goto="__end__")
return Command(
    update={"routing_log": log + ["reviewer_agent"]},
    goto="reviewer_agent",
)

```

agents/reviewer_agent.py

```

from langgraph.types import Command

MAX_REVISIONS = 3

def reviewer_agent(state):
    # your turn: judge state["draft"]; if it passes, set accepted=True and
    # go back to supervisor; if not, and revision_count < MAX_REVISIONS,
    # bump revision_count, save review_feedback, and route to writer_agent
    ...

```

Fill in `reviewer_agent`, `research_agent`, `analysis_agent`, and `writer_agent` yourself (each is small: do one job, update its field, `goto="supervisor"`), wire them into `graph.py` with `StateGraph(SharedState)`, and run a normal task through `main.py`, then go one step further, into the reasoned routing log below:

```

routing_log entries become dicts with a reason, not bare strings:
{"step": "research_agent", "reason": "no research_findings in state yet"}

reviewer keeps every rejection's feedback, not just the last one:
history = state["review_feedback_history"]
state["review_feedback_history"] = history + [feedback]

when revisions run out:
    final report includes revision_count, MAX_REVISIONS, and the full
    review_feedback_history, so a reader can see exactly what the reviewer
    kept objecting to across every attempt

graph.py:
    compile with a MemorySaver checkpointer and a thread_id, so the run
    is resumable / inspectable the same way Project 3's agent was

```

Here's almost the whole thing for the supervisor's reasoned log — fill in the missing piece yourself:

```
def supervisor(state):
    log = state["routing_log"]
    if not state.get("research_findings"):
        reason = "no research_findings in state yet"
        entry = {"step": "research_agent", "reason": reason}
        return Command(
            update={"routing_log": log + [entry]},
            goto="research_agent",
        )
    if not state.get("analysis"):
        # your turn: same shape as above, but for analysis_agent
        ...
    # ... continue the same pattern for writer_agent and reviewer_agent ...
```

Fill in the rest of `supervisor`'s reasoned routing, `reviewer_agent`'s full feedback history, and the `MemorySaver` wiring in `graph.py`, then compare all of your finished depths against the [Solution](#).

Difference between Basic and Intermediate: Basic's pseudocode names all 5 agents' jobs and the overall shape, with no code yet. Intermediate turns that into real, working `Command`-based routing and a real hard-limited reviewer loop, then adds what the Build Task's Outputs section specifically asks for and what Requirements names explicitly — a routing log that explains *why*, a "couldn't agree" report detailed enough to diagnose (not just announce), and real saved-state via a checkpoint, carried over from Project 3.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Solution

Every code block below is runnable against the `Suggested files` layout from the README. Read both depths — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

Story: get all five agents talking through one shared state, with the simplest rule-based supervisor from Core Concepts, before adding anything else. **If not:** you'd be debugging routing, logging and saved state all at once, with no working baseline to compare against.

`llm.py` — the one chat model every agent imports:

```
from langchain_openai import ChatOpenAI

# why: one place to change the model for every agent at once
model = ChatOpenAI(model="gpt-4o-mini")
```

`state.py`

```
from typing import TypedDict

class SharedState(TypedDict):
    task: str
    research_findings: str
    analysis: str
    draft: str
    review_feedback: str
    accepted: bool
    revision_count: int
    routing_log: list
```

`agents/supervisor.py`

```

from langgraph.types import Command

MAX_REVISIONS = 3

def supervisor(state):
    log = state["routing_log"]
    if not state.get("research_findings"):
        return Command(
            update={"routing_log": log + ["research_agent"]},
            goto="research_agent",
        )
    if not state.get("analysis"):
        return Command(
            update={"routing_log": log + ["analysis_agent"]},
            goto="analysis_agent",
        )
    if not state.get("draft"):
        return Command(
            update={"routing_log": log + ["writer_agent"]},
            goto="writer_agent",
        )
    if state.get("accepted"):
        return Command(goto="__end__")
    if state["revision_count"] >= MAX_REVISIONS:
        return Command(goto="__end__")
    return Command(
        update={"routing_log": log + ["reviewer_agent"]},
        goto="reviewer_agent",
    )

```

agents/research_agent.py

```

from langgraph.types import Command
from llm import model

def research_agent(state):
    prompt = f"Research this task and list key findings: {state['task']}"
    response = model.invoke(prompt)
    findings = response.content
    return Command(update={"research_findings": findings}, goto="supervisor")

```

agents/analysis_agent.py

```

from langgraph.types import Command
from llm import model

def analysis_agent(state):
    findings = state["research_findings"]
    prompt = f"Analyze what these findings mean:\n\n{findings}"
    response = model.invoke(prompt)
    return Command(update={"analysis": response.content}, goto="supervisor")

```

agents/writer_agent.py

```

from langgraph.types import Command
from llm import model

def writer_agent(state):
    feedback = state.get("review_feedback", "none yet")
    prompt = (
        f"Write a short brief for this task: {state['task']}\n\n"
        f"Analysis:\n{state['analysis']}\n\n"
        f"Reviewer feedback (if any): {feedback}"
    )
    response = model.invoke(prompt)
    return Command(update={"draft": response.content}, goto="supervisor")

```

agents/reviewer_agent.py

```

from langgraph.types import Command
from llm import model

MAX_REVISIONS = 3

def reviewer_agent(state):
    prompt = (
        "Review this draft. Reply GOOD or BAD, then a one-sentence "
        f"reason.\n\n{state['draft']}"
    )
    response = model.invoke(prompt)
    verdict = response.content.strip()

    if verdict.upper().startswith("GOOD"):
        return Command(update={"accepted": True}, goto="supervisor")

```

```

new_count = state["revision_count"] + 1
update = {
    "revision_count": new_count,
    "review_feedback": verdict,
    "accepted": False,
}
# why: while revisions are left, clearing the draft makes the
# supervisor send the task back to the Writer, with the feedback;
# at the limit, the last draft stays for the final report
if new_count < MAX_REVISIONS:
    update["draft"] = ""
return Command(update=update, goto="supervisor")

```

graph.py

```

from langgraph.graph import StateGraph, START, END
from state import SharedState
from agents.supervisor import supervisor
from agents.research_agent import research_agent
from agents.analysis_agent import analysis_agent
from agents.writer_agent import writer_agent
from agents.reviewer_agent import reviewer_agent

def build_graph():
    builder = StateGraph(SharedState)
    builder.add_node("supervisor", supervisor)
    builder.add_node("research_agent", research_agent)
    builder.add_node("analysis_agent", analysis_agent)
    builder.add_node("writer_agent", writer_agent)
    builder.add_node("reviewer_agent", reviewer_agent)
    builder.add_edge(START, "supervisor")
    return builder.compile()

```

main.py

```

from graph import build_graph

graph = build_graph()

initial_state = {
    "task": "research the benefits of remote work and write a short brief",
    "research_findings": "",
    "analysis": "",
    "draft": "",
    "review_feedback": "",
}

```

```

    "accepted": False,
    "revision_count": 0,
    "routing_log": [],
}

result = graph.invoke(initial_state)
print("Routed:", result["routing_log"])
if result["accepted"]:
    print("Final draft:\n", result["draft"])
else:
    print(f"Couldn't agree after {result['revision_count']} revisions.")

```

Expected output (LLM output varies — this is one real run):

```

Routed: ['research_agent', 'analysis_agent', 'writer_agent', 'reviewer_agent']
Final draft:
Remote work improves employee flexibility and reduces commute-related costs...

```

This works and satisfies the Build Task's basic flow. It's missing anything explaining *why* the supervisor routed the way it did beyond the bare node name, and the `graph.invoke` string `"__end__"` should really be the imported `END` constant — both fine for a first working version.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — type hints, `END` instead of a bare string, and a proper revision loop test

Story: `"__end__"` is a magic string a typo can break silently; the imported `END` can't be misspelled. And the README's "never agrees" test case deserves a real check, not an eyeball. **If not:** a misspelled end marker would quietly end nothing, and the revision limit would go untested until it mattered.

```
# agents/supervisor.py
from langgraph.graph import END
from langgraph.types import Command
from state import SharedState

MAX_REVISIONS = 3

def supervisor(state: SharedState) -> Command:
    log = state["routing_log"]
    if not state.get("research_findings"):
        return Command(
            update={"routing_log": log + ["research_agent"]},
            goto="research_agent",
        )
    if not state.get("analysis"):
        return Command(
            update={"routing_log": log + ["analysis_agent"]},
            goto="analysis_agent",
        )
    if not state.get("draft"):
        return Command(
            update={"routing_log": log + ["writer_agent"]},
            goto="writer_agent",
        )
    if state.get("accepted"):
        return Command(goto=END)
    if state["revision_count"] >= MAX_REVISIONS:
        return Command(goto=END)
    return Command(
        update={"routing_log": log + ["reviewer_agent"]},
```

```

        goto="reviewer_agent",
    )

```

This test file uses the same plain style as every exercise in this document: call the graph, `print()` what happened, and compare it against a `# expected` comment — the same pattern `convergence_and_cost_cutting` 's loop-limit test already used, not a test framework.

```

# test_project4.py -- proving the "reviewer rejects, writer improves" case
from agents.supervisor import MAX_REVISIONS
from graph import build_graph

graph = build_graph()

initial_state = {
    "task": "research the benefits of remote work and write a short brief",
    "research_findings": "",
    "analysis": "",
    "draft": "",
    "review_feedback": "",
    "accepted": False,
    "revision_count": 0,
    "routing_log": [],
}

def check_revision_loop_recovers() -> None:
    # deliberately weak first draft: writer_agent's prompt is swapped for one
    # that produces a too-short, low-effort draft on the first pass only
    result = graph.invoke(initial_state)
    print(f"routing_log length: {len(result['routing_log'])}") # >= 4
    accepted = result["accepted"]
    hit_limit = result["revision_count"] == MAX_REVISIONS
    print(f"accepted or hit the limit: {accepted or hit_limit}") # True

def check_never_agrees_hits_limit() -> None:
    # reviewer configured with an unsatisfiable bar (see
    # convergence_and_cost_cutting)
    result = graph.invoke(initial_state)
    print(f"accepted: {result['accepted']}") # False
    print(f"revision_count: {result['revision_count']}") # == MAX_REVISIONS

if __name__ == "__main__":
    check_revision_loop_recovers()
    check_never_agrees_hits_limit()

```

Difference from Basic: `Command(goto=END)` uses the real imported constant instead of the magic string `"__end__"`, which is both more correct and what autocomplete/type-checking can actually verify. `test_project4.py` now checks 2 of the README's 4 Test Cases directly (a recoverable revision, and a never-converging one hitting the limit) instead of only a manual eyeball check of printed output. Neither check yet distinguishes *why* the loop stopped in its final report, and the routing log still only names nodes, not reasons — that's what Approach 2 adds.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Approach 2 — a reasoned routing log, full rejection history, and a checkpointer

Story: the Build Task asks for a log of *why* the supervisor routed each way, and the saved state Project 3 already had. A reason next to every routing step, the full rejection history, and a checkpointer give you all three. **If not:** a bad run would show only a list of node names, and a crash would mean starting over.

state.py

```
from typing import TypedDict

class SharedState(TypedDict):
    task: str
    research_findings: str
    analysis: str
    draft: str
    review_feedback_history: list
    accepted: bool
    revision_count: int
    routing_log: list
```

agents/supervisor.py

```
from langgraph.graph import END
from langgraph.types import Command
from state import SharedState

MAX_REVISIONS = 3

def _log(state: SharedState, step: str, reason: str) -> list:
    return state["routing_log"] + [{"step": step, "reason": reason}]

def supervisor(state: SharedState) -> Command:
    if not state.get("research_findings"):
        reason = "no research_findings in state yet"
        return Command(
            update={"routing_log": _log(state, "research_agent", reason)},
            goto="research_agent",
```

```

    )
    if not state.get("analysis"):
        reason = "research_findings present, no analysis yet"
        return Command(
            update={"routing_log": _log(state, "analysis_agent", reason)},
            goto="analysis_agent",
        )
    if not state.get("draft"):
        reason = "analysis present, no draft yet"
        return Command(
            update={"routing_log": _log(state, "writer_agent", reason)},
            goto="writer_agent",
        )
    if state.get("accepted"):
        reason = "reviewer accepted the draft"
        return Command(
            update={"routing_log": _log(state, "END", reason)},
            goto=END,
        )
    # when: the hard limit – end with a clear report, never loop forever
    if state["revision_count"] >= MAX_REVISIONS:
        reason = f"revision limit ({MAX_REVISIONS}) reached without agreement"
        return Command(
            update={"routing_log": _log(state, "END", reason)},
            goto=END,
        )
    reason = "draft ready for review"
    return Command(
        update={"routing_log": _log(state, "reviewer_agent", reason)},
        goto="reviewer_agent",
    )

```

agents/reviewer_agent.py

```

from langgraph.types import Command
from agents.supervisor import MAX_REVISIONS
from llm import model
from state import SharedState

def reviewer_agent(state: SharedState) -> Command:
    prompt = (
        "Review this draft. Reply GOOD or BAD, then a one-sentence "
        f"reason.\n\n{state['draft']}"
    )
    response = model.invoke(prompt)

```

```

verdict = response.content.strip()

if verdict.upper().startswith("GOOD"):
    return Command(update={"accepted": True}, goto="supervisor")

new_count = state["revision_count"] + 1
# why: keep EVERY rejection reason, so a "couldn't agree" report
# shows the whole pattern, not just the last one
history = state["review_feedback_history"] + [verdict]
update = {
    "revision_count": new_count,
    "review_feedback_history": history,
    "accepted": False,
}
# how: under the limit, clear the draft so the Writer revises it
if new_count < MAX_REVISIONS:
    update["draft"] = ""
return Command(update=update, goto="supervisor")

```

main.py

```

from langgraph.checkpoint.memory import MemorySaver
from graph import build_graph_with_checkpointer

memory = MemorySaver()
graph = build_graph_with_checkpointer(memory)

initial_state = {
    "task": "research the benefits of remote work and write a short brief",
    "research_findings": "",
    "analysis": "",
    "draft": "",
    "review_feedback_history": [],
    "accepted": False,
    "revision_count": 0,
    "routing_log": [],
}
config = {"configurable": {"thread_id": "task-1"}}

result = graph.invoke(initial_state, config=config)

print("Routing log:")
for entry in result["routing_log"]:
    print(f"  {entry['step']}: {entry['reason']}")

if result["accepted"]:

```

```

    print("\nFinal draft:\n", result["draft"])
else:
    attempts = result["revision_count"]
    print(f"\nCouldn't agree after {attempts} revisions. Feedback history:")
    for i, feedback in enumerate(result["review_feedback_history"], start=1):
        print(f"  attempt {i}: {feedback}")

```

Expected output on a never-converging test run:

```

Routing log:
  research_agent: no research_findings in state yet
  analysis_agent: research_findings present, no analysis yet
  writer_agent: analysis present, no draft yet
  reviewer_agent: draft ready for review
  writer_agent: analysis present, no draft yet
  reviewer_agent: draft ready for review
  writer_agent: analysis present, no draft yet
  reviewer_agent: draft ready for review
  END: revision limit (3) reached without agreement

Couldn't agree after 3 revisions. Feedback history:
  attempt 1: BAD. Missing concrete examples.
  attempt 2: BAD. Still too vague in the second paragraph.
  attempt 3: BAD. Tone doesn't match the requested brief format.

```

This satisfies the README's Test Case directly: "Revision never settles... Limit is hit, a clear 'couldn't agree' report, no endless loop" — and the routing log now shows *why* the supervisor made each call, not just which node ran, satisfying the Build Task's Outputs requirement ("a visible log... and why the supervisor routed the way it did").

`graph.py`'s checkpointer wiring:

```

from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
from state import SharedState
from agents.supervisor import supervisor
from agents.research_agent import research_agent
from agents.analysis_agent import analysis_agent
from agents.writer_agent import writer_agent
from agents.reviewer_agent import reviewer_agent

def build_graph_with_checkpointer(checkpointer: MemorySaver):
    builder = StateGraph(SharedState)
    builder.add_node("supervisor", supervisor)

```

```

builder.add_node("research_agent", research_agent)
builder.add_node("analysis_agent", analysis_agent)
builder.add_node("writer_agent", writer_agent)
builder.add_node("reviewer_agent", reviewer_agent)
builder.add_edge(START, "supervisor")
return builder.compile(checkpointer=checkpointer)

```

With a checkpointer wired in, `graph.get_state(config)` can inspect the run at any point using the same `thread_id`, and a crashed or paused run can resume instead of restarting from scratch — the same saved-state guarantee Project 3's single agent had, now working across all 4 specialists and the supervisor.

Approach 3 — preventing redundant tool calls with an explicit "already ran" guard

Story: the supervisor should never send work twice, but a routing bug could — this guard makes a duplicate paid call impossible even then. **If not:** one routing mistake could double your search-API bill with nothing in the logs to explain it.

The README's Constraints specifically forbid two agents redundantly calling the same tool for the same sub-task. The supervisor's own `if not state.get(...)` checks already prevent re-running a *finished* stage, but a stricter guard is worth adding for any agent whose work involves an external tool call (like `research_agent` hitting a real search API):

```

# agents/research_agent.py
from langgraph.types import Command
from llm import model
from state import SharedState

def research_agent(state: SharedState) -> Command:
    if state.get("research_findings"):
        # already done -- supervisor shouldn't have routed here again,
        # but this guard makes it impossible even if it did
        return Command(goto="supervisor")

    prompt = f"Research this task and list key findings: {state['task']}"
    response = model.invoke(prompt)
    findings = response.content
    return Command(update={"research_findings": findings}, goto="supervisor")

```

Every specialist node (except `writer_agent`, which is *meant* to re-run on a revision) gets this same "already have my output — don't redo the work" guard at the top. This is a second, independent layer of protection beyond the supervisor's routing logic: even a routing bug that sends a task to `research_agent` twice can't cause a duplicate paid tool call.

Difference between Approach 1 and Approaches 2/3: Approach 1's routing log and test suite are functionally complete against the Requirements, but the log only names nodes and there's no persistence across runs. Approach 2 adds the reasoned log, the full rejection history, and a real checkpointer — closing the gap against the Build Task's Outputs section and its saved-state Requirement directly. Approach 3 addresses a different Constraint entirely — redundant tool calls — with a guard that's independent of (and a backstop for) the supervisor's own routing correctness.

Which one should you actually write? Approach 1 is the right bar for getting Project 4 working end to end the first time — it satisfies every Requirement functionally. Reach for Approach 2's reasoned log and full feedback history before you consider the Build Task actually *done*, since the README's own Outputs and Test Cases sections name both explicitly, not just "the pipeline finishes." Reach for Approach 3's redundant-call guard specifically once any specialist wraps a real, costed external tool (a search API, a paid model call for a sub-step) rather than just an LLM `.invoke()` — that's when a routing bug actually costs money or hits a rate limit, not just wastes a cheap extra call.